CHULETA FINAL DE JAVASCRIPT

2o DAW - Repaso completo + examen 2_2 | Metodos, sintaxis, patrones y errores tipicos

Basada en los materiales de clase aportados y en el ejercicio 2_2 (tienda.js).

OBJETIVO: Reconocer rapidamente que metodo, estructura o patron necesita cada enunciado, y escribir una solucion segura sin modificar datos cuando el ejercicio lo prohiba.

Como usar esta chuleta

- Primero mira el verbo del enunciado: transformar, filtrar, buscar, comprobar, calcular, ordenar, anadir o eliminar.
- Despues decide si necesitas un valor, un elemento, una posicion, un array nuevo o modificar el array original.
- Por ultimo comprueba mutabilidad, tipos y el valor de retorno del metodo.
- En el ejercicio 2_2, los patrones de Partes 4-6 son prioritarios.

INDICE RAPIDO POR TIPO

Tipo	Metodo	Muta?
Anadir al final	push(...)	Modifica
Quitar del final	pop()	Modifica
Anadir al inicio	unshift(...)	Modifica
Quitar del inicio	shift()	Modifica
Borrar/reemplazar	splice(i,n,...)	Modifica
Copiar parte	slice(i,f)	No
Unir arrays	concat(...)	No
Recorrer sin crear array	forEach(fn)	No
Transformar	map(fn)	No
Filtrar	filter(fn)	No
Calcular/acumular	reduce(fn, inicial)	No
Reducir derecha->izquierda	reduceRight(fn,inicial)	No
Buscar posicion	indexOf(x)	No
Buscar ultima posicion	lastIndexOf(x)	No
Buscar existencia	includes(x)	No
Buscar elemento	find(fn)	No
Buscar posicion por criterio	findIndex(fn)	No
Buscar ultima posicion por criterio	findLastIndex(fn)	No
Comprobar alguno	some(fn)	No
Comprobar todos	every(fn)	No
Ordenar numeros	sort((a,b)=>a-b)	Modifica
Ordenar texto	sort((a,b)=>a.localeCompare(b))	Modifica
Invertir	reverse()	Modifica

Tipo	Metodo	Muta?
Rellenar	fill(v)	Modifica
Aplanar	flat(n)	No
Aplanar+transformar	flatMap(fn)	No
String -> array	split(sep)	No
Array -> string	join(sep)	No
Comprobar array	Array.isArray(v)	No

MAPA DE DECISION: QUE METODO ELEGIR

Enunciado habitual	Eleccion	Esqueleto
"Transforma cada elemento"	map	array.map(x => ...)
"Quedate con los que cumplen"	filter	array.filter(x => condicion)
"Haz algo con cada uno"	forEach	array.forEach(x => ...)
"Encuentra el primero"	find	array.find(x => condicion)
"Dime su indice"	findIndex	array.findIndex(x => condicion)
"Existe este valor"	includes	array.includes(valor)
"Alguno cumple"	some	array.some(x => condicion)
"Todos cumplen"	every	array.every(x => condicion)
"Calcula un total"	reduce	array.reduce((acc,x)=>...,0)
"Ordena"	sort	[...array].sort(comparador) si no se puede mutar
"Primero de la cola"	shift	cola.shift()
"Urgente al principio"	unshift	cola.unshift(x)
"Ultimo de la pila"	pop	pila.pop()
"Anade al carrito"	push	carrito.push(x)
"Elimina por posicion"	indexOf + splice	const i=array.indexOf(x); array.splice(i,1)
"Copia un tramo"	slice	array.slice(inicio,fin)
"Texto separado por..."	split	texto.split(separador)
"Une elementos con..."	join	array.join(separador)
"Arrays anidados"	flat / flatMap	array.flat() / array.flatMap(fn)

1. FUNDAMENTOS QUE TIENES QUE TENER AUTOMATICOS

Variables, tipos y conversiones

Tema	Sintaxis / ejemplo	Recuerda
let	let edad = 18; edad = 19;	Puede reasignarse.
const	const nombre = "Miku";	No puede reasignarse; arrays/objetos pueden mutar.
typeof	typeof 42 // "number"	typeof null es "object" por una particularidad historica.
Array.isArray	Array.isArray(valor)	Es la comprobacion correcta para arrays.

Tema	Sintaxis / ejemplo	Recuerda
undefined	let x; // undefined	Ausencia de valor no inicializado.
null	let x = null;	Ausencia intencionada.
Number	Number("12") // 12	Conversion a numero.
parseInt	parseInt("12") // 12	Entero.
parseFloat	parseFloat("3.14") // 3.14	Decimal.
toString	(12).toString() // "12"	Numero -> string.

Strings y template literals

```
const nombre = "Miku"
nombre.length
const mensaje = `Producto: ${nombre}`
```

REGLA: Para construir mensajes del ejercicio usa preferentemente `...${...}...`; evita concatenaciones largas con +.

Objetos, destructuring y JSON

Concepto	Sintaxis	Uso
Objeto	const p = { nombre: "Zelda", precio: 60 };	Agrupar datos relacionados.
Propiedad	p.nombre / p[clave]	Acceder a datos.
Anadir	p.stock = 3	Agregar propiedad.
Borrar	delete p.stock	Eliminar propiedad.
Copia con cambio	{ ...p, stock: 2 }	Crear objeto nuevo y sobreescribir una propiedad.
Destructuring objeto	const { nombre, precio } = p	Extraer propiedades.
Destructuring array	const [a,b] = valores	Extraer por posicion.
Rest	const [primero, ...resto] = valores	Recoger el resto en un array.
JSON -> JS	JSON.parse(texto)	Convertir texto JSON a dato JS.
JS -> JSON	JSON.stringify(objeto)	Convertir dato JS a texto JSON.

2. ESTRUCTURAS DE CONTROL

Comparaciones y logica

Sintaxis	Significado	Ejemplo
===	Igualdad estricta	"42" === 42 // false
!==	Desigualdad estricta	stock !== 0
< > <= >=	Comparaciones	precio >= 50
&&	Y	stock > 0 && precio > 0
||	O	vip || estudiante
!	Negacion	!activo
??	Valor por defecto si null/undefined	nombre ?? "Sin nombre"
?.	Acceso seguro	usuario?.direccion?.calle

EXAMEN: Usa === y !==. Evita == y !=. Para NaN, usa Number.isNaN(x).

if / else / ternario / switch

```
if (stock > 0) {
  estado = "Disponible"
} else {
  estado = "Agotado"
}

const estado2 = stock > 0 ? "Disponible" : "Agotado"

switch (opcion) {
  case 1: ...; break
  case 2: ...; break
  default: ...
}
```

Bucles

Estructura	Cuando	Esqueleto
while	Repetir mientras se cumpla una condicion	while (condicion) { ... }
do...while	Necesitas al menos una vuelta	do { ... } while (condicion)
for	Controlar un indice/contador	for (let i=0; i<arr.length; i++) { ... }
for...of	Recorrer valores	for (const x of array) { ... }
for...in	Recorrer claves	for (const clave in objeto) { ... }
break	Salir del bucle	if (cond) break
continue	Saltar a siguiente vuelta	if (cond) continue

REGLA: for...of -> valores. for...in -> claves. En Part 6.1, piensa while + shift para procesar una cola.

3. FUNCIONES Y PROGRAMACION FUNCIONAL

Funciones

```
function sumar(a, b) {
  return a + b
}

const sumar2 = (a, b) => a + b
const doble = x => x * 2
const saludo = () => "Hola"
```

Concepto	Que significa	Ejemplo
Parametro	Nombre que recibe la funcion	function f(x) {} -> x
Argumento	Valor que pasas al llamar	f(10) -> 10
return	Devuelve y termina la funcion	return resultado
Flecha sin llaves	Return implicito	x => x * 2

Concepto	Que significa	Ejemplo
Flecha con llaves	Necesita return	x => { return x * 2 }
Valor por defecto	Se usa si llega undefined	(x = 0) => x + 1
Rest	Recoge argumentos	(primero, ...resto) => ...
Spread	Expande un array	Math.max(...numeros)

IDEA CLAVE: map, filter, find, some, every y reduce reciben funciones. Las funciones son valores de primera clase.

4. METODOS DE ARRAYS - FICHAS DE EXAMEN

push(...) - AGREGAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.push(...items)	Anadir al final.	Nueva longitud	SI

```
carrito.push("Zelda")
```

USO CONJUNTO: push -> cola; push + shift = FIFO.

ERROR TIPICO: Confundir push con unshift.

pop() - QUITAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.pop()	Quitar el ultimo.	Elemento eliminado / undefined	SI

```
const ultimo = pila.pop()
```

USO CONJUNTO: push + pop = pila LIFO.

ERROR TIPICO: No usar para quitar el primero.

unshift(...) - AGREGAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.unshift(...items)	Anadir al principio.	Nueva longitud	SI

```
cola.unshift(pedidoUrgente)
```

USO CONJUNTO: unshift + pop tambien puede implementar una cola.

ERROR TIPICO: Usarlo cuando el enunciado diga urgente/primero.

shift() - QUITAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.shift()	Quitar el primero.	Elemento eliminado / undefined	SI

```
const siguiente = cola.shift()
```

USO CONJUNTO: En 5.1 y 6.1: cola FIFO.

ERROR TIPICO: No confundir con pop().

splice(i,n,...) - EDITAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.splice(indice, cuantos, ...nuevos)	Borrar, insertar o reemplazar en el medio.	Array con lo eliminado	SI

```
carrito.splice(posicion, 1)
arr.splice(2, 0, "nuevo")
```

USO CONJUNTO: indexOf + splice = eliminar la primera aparicion.

ERROR TIPICO: splice modifica el original; slice no.

slice(i,f) - COPIAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.slice(inicio, fin)	Copiar un tramo o hacer una copia completa.	Array nuevo	NO

```
const copia = array.slice()
const trozo = array.slice(1, 3)
```

USO CONJUNTO: slice + sort permite ordenar una copia.

ERROR TIPICO: El fin NO se incluye.

concat(...) - UNIR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.concat(...items)	Unir arrays sin mutar.	Array nuevo	NO

```
const todos = a.concat(b)
```

USO CONJUNTO: Alternativa: [...a, ...b].

ERROR TIPICO: concat no modifica los originales.

5. METODOS DE BUSQUEDA

indexOf(x) - BUSQUEDA

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.indexOf(valor)	Saber la primera posicion de un valor concreto.	Indice o -1	NO

```
const i = carrito.indexOf("Zelda")
```

USO CONJUNTO: indexOf + splice para borrar una aparicion.

ERROR TIPICO: Busca por valor concreto; usa find para objetos/criterios.

lastIndexOf(x) - BUSQUEDA

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.lastIndexOf(valor)	Saber la ultima posicion de un valor concreto.	Indice o -1	NO

```
historial.lastIndexOf("añadir")
```

USO CONJUNTO: Muy util en deshacer cuando hay repetidos.

ERROR TIPICO: No devuelve el elemento.

includes(x) - BUSQUEDA

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.includes(valor)	Saber solo si un valor existe.	true / false	NO

```
["Ana","Luis"].includes("Luis")
```

USO CONJUNTO: Tambien puede comprobar existencia tras map.

ERROR TIPICO: No sirve para buscar con una condicion.

find(fn) - BUSQUEDA

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.find(x => condicion)	Encontrar el primer elemento que cumple.	Elemento o undefined	NO

```
catalogo.find(p => p.nombre === "Zelda")
```

USO CONJUNTO: find + propiedad; find para luego leer/modificar un objeto encontrado.

ERROR TIPICO: Puede devolver undefined: no accedas a propiedades sin comprobar.

findIndex(fn) - BUSQUEDA

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.findIndex(x => condicion)	Encontrar la posicion del primer elemento que cumple.	Indice o -1	NO

```
const i = catalogo.findIndex(p => p.nombre === "Zelda")
```

USO CONJUNTO: findIndex + splice para eliminar por criterio.

ERROR TIPICO: Si no existe, devuelve -1.

findLastIndex(fn) - BUSQUEDA

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.findLastIndex(x => condicion)	Buscar la ultima posicion que cumple un criterio.	Indice o -1	NO

```
historial.findLastIndex(x => x.tipo === "quitar")
```

USO CONJUNTO: Alternativa especializada cuando se quiere la ultima coincidencia.

ERROR TIPICO: Es una extension de 02.2, no el patron central de tienda.js.

6. TRANSFORMAR, FILTRAR Y CALCULAR

map(fn) - TRANSFORMAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.map(x => resultado)	Crear un array nuevo transformando cada elemento.	Array nuevo del mismo tamaño	NO

```
catalogo.map(p => p.nombre)
```

USO CONJUNTO: filter(...).map(...), map(...).join("\n"), map(...).flat()

ERROR TIPICO: No es para filtrar: siempre genera una salida por elemento.

filter(fn) - FILTRAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.filter(x => condicion)	Quedarte con todos los elementos que cumplen.	Array nuevo	NO

```
catalogo.filter(p => p.stock > 0)
```

USO CONJUNTO: filter + map es el patron mas tipico.

ERROR TIPICO: No devuelve un unico objeto.

forEach(fn) - RECORRER

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.forEach((x,i,arr) => ...)	Ejecutar una accion por elemento sin crear resultados.	undefined	NO*

```
productos.forEach(p => console.log(p.nombre))
```

USO CONJUNTO: Usalo cuando el resultado es un efecto (console.log, push, etc.).

ERROR TIPICO: Si necesitas un array, usa map; forEach devuelve undefined.

reduce(fn,inicial) - CALCULO

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.reduce((acc, x) => ..., inicial)	Acumular hasta obtener un unico valor.	Un valor	NO

```
const total = productos.reduce((s,p) => s + p.precio, 0)
```

USO CONJUNTO: reduce + find; reduce para totales; reduce con objeto acumulador.

ERROR TIPICO: Especifica valor inicial; sin el, [] produce error.

reduceRight(fn,inicial) - CALCULO

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.reduceRight((acc,x) => ..., inicial)	Reducir empezando por la derecha.	Un valor	NO

```
["a","b","c"].reduceRight((t,x) => t+x, "")
```

USO CONJUNTO: 02.2 lo presenta como variante de reduce.

ERROR TIPICO: No confundir con reverse + reduce.

some(fn) - COMPROBACION

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.some(x => condicion)	Comprobar si al menos uno cumple.	true / false	NO

```
catalogo.some(p => p.stock === 0)
```

USO CONJUNTO: some = "hay alguno"; ideal para hayAgotados.

ERROR TIPICO: No significa todos.

every(fn) - COMPROBACION

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.every(x => condicion)	Comprobar si todos cumplen.	true / false	NO

```
pedido.lineas.every(linea => linea.cantidad > 0)
```

USO CONJUNTO: every = "todos"; clave en puedeServirse y preciosValidos.

ERROR TIPICO: Asegurate de recorrer el array correcto.

7. ORDENAR Y METODOS EXTRA

sort(fn) - ORDENAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.sort((a,b) => ...)	Ordenar elementos.	Array ordenado	SI

```
[...productos].sort((a,b) => a.precio - b.precio)
```

USO CONJUNTO: Si el enunciado no permite mutar: [...array].sort(...).

ERROR TIPICO: Sin comparador, los numeros se comparan como texto.

localeCompare - ORDENAR TEXTO

Sintaxis	Cuando usarlo	Que devuelve	Muta?
a.localeCompare(b)	Ordenar cadenas respetando el idioma/acento.	Numero <0, 0, >0	NO

```
productos.sort((a,b) => a.nombre.localeCompare(b.nombre))
```

USO CONJUNTO: sort + localeCompare para nombres.

ERROR TIPICO: No uses a > b si el ejercicio habla de orden alfabetico con acentos.

reverse() - ORDENAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.reverse()	Invertir el orden.	Array invertido	SI

```
[...array].reverse()
```

USO CONJUNTO: sort ascendente + reverse para descendente, si el ejercicio lo plantea.

ERROR TIPICO: Tambien muta: copia primero si hace falta.

fill(v,i,f) - RELLENAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.fill(valor, inicio, fin)	Rellenar posiciones.	Array	SI

```
new Array(5).fill("■")
```

USO CONJUNTO: En graficoStock: new Array(n).fill("■").join("").

ERROR TIPICO: Modifica el array.

flat(n) - APLANAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.flat(profundidad)	Aplanar arrays anidados.	Array nuevo	NO

```
[["A","B"],["C"]].flat()
```

USO CONJUNTO: map + flat aparece en cadenas de procesamiento.

ERROR TIPICO: flat() por defecto aplana un nivel.

flatMap(fn) - APLANAR + TRANSFORMAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.flatMap(x => resultadoArray)	Transformar y aplanar un nivel en una sola operacion.	Array nuevo	NO

```
["hola mundo"].flatMap(x => x.split(" "))
```

USO CONJUNTO: Alternativa compacta a map(...).flat().

ERROR TIPICO: Aplana un solo nivel.

split(sep) - STRING -> ARRAY

Sintaxis	Cuando usarlo	Que devuelve	Muta?
texto.split(separador)	Separar una cadena por un delimitador.	Array	NO

```
"Ana|Pan:2;Leche:1".split("|")
```

USO CONJUNTO: 4.1: split("|") -> split(";") -> split(":").

ERROR TIPICO: split siempre trabaja sobre strings.

join(sep) - ARRAY -> STRING

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.join(separador)	Unir elementos en un texto.	String	NO

```
["Pan","Leche"].join("\n")
```

USO CONJUNTO: 4.5: map + join("\n") para generar ticket.

ERROR TIPICO: No confundir con split.

Array.isArray(v) - TIPO

Sintaxis	Cuando usarlo	Que devuelve	Muta?
Array.isArray(valor)	Comprobar si un valor es un array.	true / false	NO

```
Array.isArray(matriz)
```

USO CONJUNTO: 1.1 y validaciones.

ERROR TIPICO: typeof [] devuelve "object".

toSorted(fn) - ORDENAR SIN MUTAR

Sintaxis	Cuando usarlo	Que devuelve	Muta?
array.toSorted(comparador)	Ordenar sin tocar el original (ES2023).	Array nuevo	NO

```
const ordenados = numeros.toSorted((a,b) => a-b)
```

USO CONJUNTO: Alternativa moderna a [...array].sort(...).

ERROR TIPICO: Tu ejercicio 2_2 trabaja con ES6 y puede exigir spread/slice.

8. TABLA VITAL: MUTA O NO MUTA

Categoria	Metodos / patron
SI mutan	push, pop, shift, unshift, splice, sort, reverse, fill, copyWithin
NO mutan	slice, concat, map, filter, reduce, reduceRight, find, findIndex, findLastIndex, indexOf, lastIndexOf, includes, some, every, join, flat, flatMap
Copia tipica	[...array] o array.slice()
Objeto copia	{ ...objeto, propiedad: nuevoValor }

EXAMEN 2_2: Excepto Parte 5, las funciones no deben modificar los arrays de entrada. Para ordenar, copia antes con [...array] o slice().

9. CUANDO USAR METODOS JUNTOS: RECETAS

Combinacion	Idea	Patron	Uso en examen
filter + map	Seleccionar y despues transformar	productos.filter(p => p.stock > 0).map(p => p.nombre)	Nombres de productos con stock.
map + join	Transformar y construir texto	lineas.map(l => `${l.nombre}: ${l.cantidad}`).join("\n")	Ticket / listado.
split + map	Parsear una cadena a datos	texto.split(";").map(x => x.split(":"))	Pedidos.
find + propiedad	Buscar un objeto y usar un dato	const p = catalogo.find(x => x.nombre === nombre); p.precio	Buscar precio/stock.
findIndex + splice	Eliminar el primer elemento que coincide	const i = a.findIndex(x => x.id === id); if (i !== -1) a.splice(i,1)	Borrar por criterio.
indexOf + splice	Eliminar un valor concreto	const i = a.indexOf(valor); if (i !== -1) a.splice(i,1)	Carrito Parte 5.
reduce + find	Acumular usando datos de objetos	lineas.reduce((t,l) => t + buscarProducto(catalogo,l.nombre).precio*l.cantidad, 0)	Total de pedido.
map + flat + filter + indexOf + sort	Transformar, aplanar, quitar repetidos y ordenar	pedidos.map(p => p.lineas).flat().map(l => l.nombre).filter((n,i,a) => a.indexOf(n) === i).sort((a,b)=>a.localeCompare(b))	productosVendidos 6.2.
[...] + sort	Ordenar sin mutar	[...catalogo].sort((a,b) => a.precio-b.precio)	ordenarPorPrecio / 3 mas baratos.
while + shift	Procesar una cola hasta vaciarla	while (cola.length > 0) { const p = cola.shift(); ... }	procesarCola 6.1.
new Array + fill + join	Crear una barra visual	new Array(Math.floor(stock/2)).fill("■").join("")	graficoStock 6.3.

10. EJERCICIO 2_2 - GUIA DE PATRONES POR PARTES

PRIORIDAD MAXIMA: Partes 4, 5 y 6. Aprende el patron, no memorices la solucion completa.

Parte 4 - Pedidos

Funcion	Que pide	Patron mental	Metodos clave
4.1 parsearPedido	Texto -> objeto pedido	capas de split + map + Number	split, map, Number
4.2 puedeServirse	Todos existen y tienen stock suficiente	pedido.lineas.every -> buscarProducto -> stock >= cantidad	every, find
4.3 totalPedido	Total euros	reduce -> buscarProducto -> precio*cantidad	reduce, find
4.4 servirPedido	Nuevo catalogo con stock descontado	catalogo.map -> find linea -> { ...producto, stock: ... }	map, find, spread
4.5 generarTicket	Texto final con saltos de linea	map -> template literal -> join("\n")	map, join, template literal

```
// 4.1 esquema
const partes = texto.split("|")
const cliente = partes[0]
const lineas = partes[1].split(";").map(linea => {
  const p = linea.split(":")
  return { nombre: p[0], cantidad: Number(p[1]) }
})
```

```
// 4.2 esquema
return pedido.lineas.every(linea => {
  const producto = buscarProducto(catalogo, linea.nombre)
  return producto !== undefined && producto.stock >= linea.cantidad
})

// 4.3 esquema
return pedido.lineas.reduce((total, linea) => {
  const producto = buscarProducto(catalogo, linea.nombre)
  return total + producto.precio * linea.cantidad
}, 0)
```

Parte 5 - Arrays mutables

Funcion	Idea	Metodo	Patron
5.1 atenderSiguiente	Sale el primero	shift	const pedido = cola.shift()
5.2 agregarUrgente	Entra por delante	unshift	cola.unshift(pedido)
5.3 agregarAlCarrito	Entra al final + historial	push	carrito.push(producto)
5.4 quitarDelCarrito	Buscar primera posicion y borrar	indexOf + splice	const i = carrito.indexOf(x); if (i !== -1) carrito.splice(i,1)
5.5 deshacer	Deshacer ultima accion	pop + reverse action + lastIndexOf/splice	const accion = historial.pop(); ...

PISTA: Parte 5 trata al historial como una pila LIFO: la ultima accion registrada es la primera que se deshace.

Parte 6 - Integracion

Funcion	Secuencia	Clave
6.1 procesarCola	while -> shift -> puedeServirse -> servirPedido -> reasignar catalogo	Cola + condicion + inmutabilidad
6.2 productosVendidos	map -> flat -> map -> filter + indexOf -> sort + localeCompare	Pipelines encadenados
6.3 graficoStock	calcular n -> new Array(n).fill("■").join("")	String visual desde un array

```
// 6.1 patron
while (cola.length > 0) {
  const pedido = atenderSiguiente(cola)
  if (puedeServirse(catalogo, pedido)) {
    catalogo = servirPedido(catalogo, pedido)
  }
}
```

11. CHECKLIST DE EXAMEN: ANTES DE ENTREGAR

- [ ] He leido exactamente que pide el enunciado y que array debo recorrer.
- [ ] He elegido map/filter/find/findIndex/some/every/reduce segun el verbo.
- [ ] He usado === y !==, no ==/!=.
- [ ] He comprobado undefined cuando find puede no encontrar nada.
- [ ] He comprobado -1 cuando uso indexOf/findIndex antes de splice.
- [ ] He usado Number() cuando un numero viene de split().
- [ ] He usado localeCompare para texto cuando el orden alfabetico importa.
- [ ] He puesto comparador numerico en sort((a,b)=>a-b).
- [ ] He copiado antes de sort/reverse si no puedo mutar la entrada.

- [ ] He recordado que const no congela arrays/objetos.
- [ ] He usado shift para sacar el primero y push/unshift segun donde entra.
- [ ] He distinguido slice (copia) de splice (modifica).
- [ ] He distinguido split (string -> array) de join (array -> string).
- [ ] En flechas con llaves he puesto return.
- [ ] En reduce he puesto un valor inicial, normalmente 0 para sumas.
- [ ] En 4.2 recorro pedido.lineas, no catalogo.
- [ ] En 4.4 creo objetos nuevos con {...producto, stock: ...}.
- [ ] En 6.1 uso while(cola.length > 0).
- [ ] En 6.2 reconozco la cadena map/flat/map/filter/indexOf/sort.
- [ ] No he cambiado nombres ni parametros que el profesor haya fijado.

12. LA SUPER-CHULETA DE 30 SEGUNDOS

Pregunta del enunciado	Respuesta
Transformar cada elemento	map
Seleccionar varios	filter
Hacer algo por cada uno	forEach
Buscar un elemento	find
Buscar una posicion	findIndex
Buscar si existe	includes
Existe alguno	some
Cumplen todos	every
Calcular un total	reduce
Ordenar numeros	sort((a,b)=>a-b)
Ordenar textos	sort((a,b)=>a.localeCompare(b))
Anadir final	push
Quitar final	pop
Anadir inicio	unshift
Quitar inicio	shift
Eliminar por posicion	splice
Copiar	slice / [...array]
Texto -> array	split
Array -> texto	join
Array -> array plano	flat
Copia objeto + cambio	{ ...objeto, prop: valor }
Todos los pedidos hasta vaciar cola	while + shift
Ultima accion primero	pop (LIFO)
Primero que llega primero sale	shift (FIFO)

MIKU: No intentes recordar 50 soluciones distintas. Reconoce 15-20 patrones y combina los metodos que ya conoces.

Documento de estudio elaborado a partir de los materiales aportados para el repaso del examen.

AMPLIACION: ejemplos de clase + como construir las soluciones

Estas paginas se anaden a la chuleta original: las 15 paginas anteriores se conservan sin cambios. Aqui se incorpora lo que faltaba para estudiar mejor: ejemplos de ejercicios, resultado esperado, patrones de combinacion y una lectura guiada de las funciones.

INDICE DE LA AMPLIACION

A. Como leer y escribir una funcion
B. Como crear y recorrer arrays y objetos
C. Tabla maestra: que pregunta responde cada metodo
D. Agregar, quitar, slice y splice con ejemplos
E. Busqueda y validacion
F. Transformacion: map, filter, forEach
G. Calculo: reduce y reduceRight
H. Ordenar, split, join y flat
I. Metodos extra del nuevo tema 2.2
J. Mutabilidad, spread, copias y metodos encadenados
K. Ejemplos exactos del ejercicio 2.2 - Parte 4
L. Ejemplos exactos del ejercicio 2.2 - Parte 5
M. Ejemplos exactos del ejercicio 2.2 - Parte 6
N. Patrones de examen y errores que cuestan puntos

A. COMO CREAR UNA FUNCION Y COMO LEERLA

Idea: una funcion recibe datos, hace un trabajo y puede devolver un resultado. En tu examen casi todas aparecen como funciones flecha y con return cuando hay que devolver algo.

```
const sumar = (a, b) => { return a + b }
```

Como leerla: sumar es el nombre; a, b son parametros; en la llamada sumar(2, 3), 2 y 3 son argumentos; return devuelve 5.

```
const resultado = sumar(2, 3) // resultado -> 5
```

Flecha corta: cuando solo devuelves una expresion, puedes quitar llaves y return.

```
const doble = n => n * 2 // doble(5) -> 10
```

Con bloque: si pones llaves, necesitas return para devolver un valor.

```
const doble = n => { const resultado = n * 2 return resultado }
```

Funcion que procesa un array: aqui aparece la idea que luego usan map, filter, find, every, etc.: pasas una funcion como argumento.

```
const mayores = numeros => numeros.filter(n => n > 10)
```

Resultado: mayores([4, 12, 20, 3]) -> [12, 20]

B. COMO CREAR ARRAYS, OBJETOS Y ARRAYS DE OBJETOS

Array: una lista ordenada. Los indices empiezan en 0. length indica la cantidad de elementos.

```
const juegos = ['Zelda', 'Persona', 'Mario'] juegos[0] // 'Zelda' juegos.length // 3 juegos[juegos.length - 1] // 'Mario'
```

Objeto: un conjunto de propiedades nombre/valor.

```
const producto = { nombre: 'Zelda', precio: 60, stock: 4 } producto.nombre producto['precio']
```

Array de objetos: es la estructura central de tienda.js. Piensa siempre: primero el array, despues cada objeto y despues la propiedad.

```
const catalogo = [ { nombre: 'Pan', precio: 2, stock: 5 }, { nombre: 'Leche', precio: 1.5, stock: 0 } ]
catalogo[0].nombre // 'Pan'
```

Patron al usar metodos: en map, filter, find, etc., el parametro suele llamarse producto o linea y representa el elemento actual.

```
catalogo.map(producto => producto.nombre) // ['Pan', 'Leche']
```

C. TABLA MAESTRA: QUE PREGUNTA RESPONDE CADA METODO

Pregunta del enunciado	Metodo	Sintaxis base	Resultado
Transforma cada elemento	map	arr.map(x => ...)	array nuevo
Quedate con los que...	filter	arr.filter(x => condicion)	array nuevo
Haz algo por cada uno	forEach	arr.forEach(x => ...)	undefined
Busca un elemento	find	arr.find(x => condicion)	elemento/undefined
Busca su posicion	findIndex	arr.findIndex(x => condicion)	indice/-1
Busca desde el final	findLastIndex	arr.findLastIndex(x => condicion)	indice/-1
Existe ese valor	includes	arr.includes(valor)	true/false
Primera posicion	indexOf	arr.indexOf(valor)	indice/-1
Ultima posicion	lastIndexOf	arr.lastIndexOf(valor)	indice/-1
Hay alguno	some	arr.some(x => condicion)	true/false
Todos	every	arr.every(x => condicion)	true/false
Calcula un unico resultado	reduce	arr.reduce((acc,x)=>..., inicial)	valor
Reduce de derecha a izquierda	reduceRight	arr.reduceRight((acc,x)=>..., inicial)	valor
Ordena	sort	arr.sort((a,b)=>a-b)	array ordenado (muta)
Invierte	reverse	arr.reverse()	array invertido (muta)
Copia un tramo	slice	arr.slice(inicio, fin)	array nuevo
Borra/inserta en una posicion	splice	arr.splice(i,n,...)	array modificado
Une arrays	concat	a.concat(b)	array nuevo
Array -> texto	join	arr.join(separador)	string
Texto -> array	split	texto.split(separador)	array
Aplana arrays anidados	flat	arr.flat(profundidad)	array nuevo
Rellena posiciones	fill	arr.fill(valor,inicio,fin)	array modificado

Atajo mental: map = transformar; filter = seleccionar; find = uno; findIndex = posicion; some = alguno; every = todos; reduce = total; sort = ordenar. Esta traduccion es exactamente la que mas te interesa en el examen.

D. AGREGAR, QUITAR, SLICE Y SPLICE

Los cuatro extremos: push/unshift agregan; pop/shift quitan. push/pop trabajan al final; unshift/shift al principio.

```
const cola = ['Ana', 'Luis'] cola.push('Eva') // ['Ana','Luis','Eva'] cola.unshift('Marta') //
['Marta','Ana','Luis','Eva'] const ultimo = cola.pop() // 'Eva' const primero = cola.shift() // 'Marta'
```

splice: "cirugia". Sirve para borrar, insertar o reemplazar. Su firma es splice(indice, cuantos, ...nuevos).

```
const a = ['Yo', 'estudio', 'JS'] a.splice(1, 1) // ['Yo', 'JS'] a.splice(1, 0, 'aprendo') // ['Yo', 'aprendo', 'JS']
```

slice: "fotocopia". No modifica el original y el fin NO se incluye.

```
const a = ['A', 'B', 'C', 'D'] const parte = a.slice(1, 3) // ['B','C'] // a sigue intacto
```

Ejercicio de tienda: para quitar la primera aparicion de un elemento del carrito, busca su posicion con indexOf y despues usa splice.

```
const pos = carrito.indexOf('Zelda') if (pos !== -1) { carrito.splice(pos, 1) }
```

E. BUSQUEDA Y VALIDACION

Arrays simples: indexOf/lastIndexOf/includes trabajan directamente con valores. Arrays de objetos: find/findIndex/findLastIndex reciben una condicion.

```
const nombres = ['Ana', 'Luis', 'Ana'] nombres.indexOf('Ana') // 0 nombres.lastIndexOf('Ana') // 2
nombres.includes('Luis') // true
```

```
const usuarios = [ { id: 1, nombre: 'Ana' }, { id: 2, nombre: 'Luis' } ] usuarios.find(u => u.id === 2) // { id: 2,
nombre: 'Luis' } usuarios.findIndex(u => u.id === 2) // 1
```

some: responde "hay al menos uno?". every: responde "todos cumplen?".

```
const stock = [0, 3, 5] stock.some(s => s === 0) // true stock.every(s => s >= 0) // true
```

Aplicacion directa al ejercicio: hayAgotados -> some; preciosValidos -> every.

Regla: si necesitas el objeto para seguir calculando algo, find suele ser mejor que includes; si solo quieres un booleano sobre un valor literal, includes es mas directo.

F. TRANSFORMACION: MAP, FILTER Y FOREACH

map mantiene el numero de elementos y cambia lo que representa cada posicion.

```
const precios = [10, 20, 30] const iva = precios.map(p => p * 1.21) // [12.1, 24.2, 36.3]
```

Ejercicio 1 de arrays: playlist -> forEach para imprimir titulo y artista. No necesitas crear otro array porque solo quieres realizar un efecto.

```
canciones.forEach(cancion => { console.log(`${cancion.titulo} - ${cancion.artista}`) })
```

Ejercicio 2 de arrays: primero filtras canciones de mas de 180 s y despues transformas cada una en string.

```
const resultado = canciones .filter(c => c.duracion > 180) .map(c => `${c.titulo} - ${c.duracion}s`)
```

Ejemplo de tienda: agotados = filter para quedarnos con stock 0 + map para obtener solo nombres.

```
const nombres = catalogo .filter(p => p.stock === 0) .map(p => p.nombre) // ['Leche', 'Huevos']
```

ERROR TIPICO: usar forEach cuando el enunciado pide un array resultante. forEach devuelve undefined; map devuelve un array nuevo.

G. CALCULO: REDUCE Y REDUCERIGHT

reduce convierte muchos elementos en un unico resultado. Para el examen piensa en: total, suma, promedio con acumulador, maximo, agrupacion.

```
const numeros = [1, 2, 3, 4] const total = numeros.reduce((acum, n) => acum + n, 0) // 10
```

Valor inicial: en tu nuevo tema se recomienda especificarlo siempre, especialmente porque un array vacio sin valor inicial produce error.

```
[].reduce((a, b) => a + b, 0) // 0
```

Ejercicio 2.2 - valorAlmacen: cada producto aporta precio * stock.

```
const total = catalogo.reduce((acum, producto) => { return acum + producto.precio * producto.stock }, 0)
```

Ejercicio 2.2 - productoMasCaro: reduce compara dos objetos y conserva el de mayor precio.

```
const masCaro = catalogo.reduce((mejor, p) => { if (mejor === undefined || p.precio > mejor.precio) { return p }
return mejor }, undefined)
```

ReduceRight: misma idea, recorriendo desde la derecha. Ejemplo: convertir [a,b,c] en cba.

```
['a', 'b', 'c'].reduceRight((texto, letra) => texto + letra, '') // 'cba'
```

H. ORDENAR, SPLIT, JOIN Y FLAT

sort numerico: siempre usa comparador cuando ordenas numbers.

```
const nums = [10, 2, 15] nums.sort((a, b) => a - b) // [2,10,15] nums.sort((a, b) => b - a) // [15,10,2]
```

Texto: localeCompare. En tus ejercicios, nombresOrdenados usa map + sort + localeCompare.

```
const nombres = catalogo .map(p => p.nombre) .sort((a, b) => a.localeCompare(b))
```

split/join: 4.1 de tienda usa split para romper el texto del pedido; 4.5 usa join para volver a construir el ticket.

```
'Ana|Pan:2;Leche:1'.split('|') // ['Ana', 'Pan:2;Leche:1'] ['Pan x2', 'Leche x1'].join('
') // 'Pan x2
Leche x1'
```

flat: si tienes arrays dentro de arrays y quieres un solo nivel.

```
const lineas = pedidos.map(p => p.lineas) const todas = lineas.flat()
```

Combinacion muy frecuente: map -> flat -> map -> filter -> sort. Eso es exactamente el tipo de cadena que aparece en productosVendidos.

I. METODOS EXTRA DEL NUEVO TEMA 2.2

Metodo	Sintaxis	Uso	Ejemplo / resultado
findLastIndex	arr.findLastIndex(x => cond)	Ultima posicion que cumple una condicion	usuarios.findLastIndex(u => u.nombre === 'Celina') ->
reduceRight	arr.reduceRight((acc,x)=>..., inicial)	Acumular desde la derecha	[a,b,c] -> cba
fill	arr.fill(valor, inicio, fin)	Rellenar una zona del array; muta	[1,2,3,4].fill(9,1,3) -> [1,9,9,4]
flat	arr.flat(profundidad)	Aplanar arrays anidados	[[1,2],[3]] .flat() -> [1,2,3]
flatMap	arr.flatMap(fn)	Map + un nivel de flat en una sola pasada	['hola mundo'].flatMap(s => s.split(' ')) -> ['hola','mundo
toSorted	arr.toSorted(fn)	Ordenar sin mutar (ES2023)	ordenada = original.toSorted((a,b)=>a-b)
Array.isArray	Array.isArray(v)	Comprobar si v es array	Array.isArray([]) -> true

Prioridad para tu examen: los ejercicios que me pasaste trabajan sobre todo map/filter/reduce/find/findIndex/some/every/sort/localeCompare/push/pop/shift/unshift/splice/slice/indexOf/lastIndexOf/includes/join/split/flat. Los metodos extra entran en tu nuevo tema, asi que conviene reconocer su sintaxis aunque no sean el nucleo de 2.2.

J. MUTABILIDAD, COPIAS Y ENCADENAMIENTO

Mutan: push, pop, shift, unshift, splice, sort, reverse, fill, copyWithin. No mutan: slice, concat, map, filter, find, findIndex, includes, reduce, join, flat, some, every.

Regla del ejercicio 2.2: si la funcion no debe tocar el array de entrada y necesitas ordenar, copia primero.

```
const ordenados = [...productos].sort((a, b) => a.precio - b.precio)
```

Objeto nuevo con spread: util para servirPedido, porque cambias stock sin modificar el objeto de entrada.

```
const nuevoProducto = { ...producto, stock: producto.stock - cantidad }
```

Encadenamiento: puedes usar el resultado de un metodo como entrada del siguiente. Lee de izquierda a derecha.

```
const baratos = productos .filter(p => p.precio < 50) .map(p => p.nombre) // primero selecciona; despues transforma
```

Otro patron: map -> flat -> map -> filter -> sort. No lo memorices como una formula; memoriza que cada paso tiene una finalidad.

K. EJERCICIO 2.2 - PARTE 4: PEDIDOS

4.1 parsearPedido - ejemplo completo con el formato real del ejercicio.

```
const texto = 'Ana|Pan:2;Leche:1' const partes = texto.split('|') const cliente = partes[0] const lineas =
partes[1].split(';').map(linea => { const datos = linea.split(':') return { nombre: datos[0], cantidad:
Number(datos[1]) } }) // { cliente: 'Ana', lineas: [ // {nombre:'Pan', cantidad:2}, {nombre:'Leche', cantidad:1} ] }
```

4.2 puedeServirse - "TODOS" -> every. Hay que recorrer pedido.lineas, buscar cada producto y comparar stock con cantidad.

```
return pedido.lineas.every(linea => { const producto = buscarProducto(catalogo, linea.nombre) return producto !==
undefined && producto.stock >= linea.cantidad })
```

4.3 totalPedido - "TOTAL" -> reduce; cada linea aporta precio * cantidad.

```
return pedido.lineas.reduce((total, linea) => { const producto = buscarProducto(catalogo, linea.nombre) return total +
producto.precio * linea.cantidad }, 0)
```

4.4 servirPedido - "catalogo nuevo" -> map; encuentra la linea y devuelve un objeto con spread.

```
return catalogo.map(producto => { const linea = pedido.lineas.find(l => l.nombre === producto.nombre) if (linea ===
undefined) return producto return { ...producto, stock: producto.stock - linea.cantidad } })
```

4.5 generarTicket - cada linea se transforma en texto y despues se unen con salto de linea.

```
return pedido.lineas .map(linea => { const producto = buscarProducto(catalogo, linea.nombre) return `${linea.nombre}
x${linea.cantidad} = ${producto.precio * linea.cantidad}€` }) .join('
')
```

L. EJERCICIO 2.2 - PARTE 5: ARRAYS MUTABLES

5.1 atenderSiguiente - una cola es FIFO: sale el primero -> shift.

```
export const atenderSiguiente = cola => cola.shift()
```

5.2 agregarUrgente - entra por delante -> unshift.

```
export const agregarUrgente = (cola, pedido) => { cola.unshift(pedido) return cola }
```

5.3 agregarAlCarrito - entra por el final -> push. Si hay historial, registra la accion.

```
carrito.push(producto) historial.push({ tipo: 'agregar', producto })
```

5.4 quitarDelCarrito - primero encuentra la posicion y luego splice(pos, 1).

```
const pos = carrito.indexOf(producto) if (pos !== -1) { carrito.splice(pos, 1) historial.push({ tipo: 'quitar',
producto }) }
```

5.5 deshacer - el historial es una pila LIFO: la ultima accion es la primera que se revierte.

```
const accion = historial.pop() // si fue 'agregar' -> quitar el producto // si fue 'quitar' -> volver a insertarlo //
lastIndexOf ayuda si hay varias copias del mismo producto
```

Reconocimiento instantaneo: cola = shift/unshift; pila/historial = pop/push.

M. EJERCICIO 2.2 - PARTE 6: INTEGRACION

6.1 procesarCola combina while + atenderSiguiente + puedeServirse + servirPedido.

```
while (cola.length > 0) { const pedido = atenderSiguiente(cola) if (puedeServirse(catalogo, pedido)) { catalogo = servirPedido(catalogo, pedido) } }
```

6.2 productosVendidos exige varias etapas. Una forma de leer el problema es: sacar lineas -> aplanar -> sacar nombres -> quitar repetidos -> ordenar.

```
const nombres = pedidos .map(p => p.lineas) .flat() .map(linea => linea.nombre) .filter((nombre, i, arr) => arr.indexOf(nombre) === i) .sort((a, b) => a.localeCompare(b))
```

6.3 graficoStock crea una cadena proporcional al stock. La idea es combinar new Array + fill + join.

```
const barras = new Array(stock).fill('■').join('') // stock = 5 -> '■■■■■'
```

Truco: si una funcion de la parte 6 parece demasiado larga, dividela mentalmente en mini pasos y asigna cada verbo a un metodo.

N. PATRONES DE EXAMEN Y ERRORES QUE CUESTAN PUNTOS

Cuando leas...	Piensa en...	Ejemplo
"TODOS"	every	pedido.lineas.every(...)
"ALGUNO / HAY ALGUNO"	some	catalogo.some(p => p.stock === 0)
"BUSCAR PRODUCTO"	find	catalogo.find(p => ...)
"POSICION"	findIndex / indexOf	findIndex para objetos; indexOf para valor
"NOMBRE DE CADA"	map	catalogo.map(p => p.nombre)
"SOLO LOS QUE"	filter	catalogo.filter(p => p.stock > 0)
"TOTAL / SUMA"	reduce	reduce((acc,p)=>acc+p.precio,0)
"ORDEN ALFABETICO"	sort + localeCompare	sort((a,b)=>a.localeCompare(b))
"NO MODIFICAR"	copia con spread/slice	[...array].sort(...)
"COLA"	shift / unshift	sale el primero
"HISTORIAL / DESHACER"	pop / push	sale la ultima accion
"QUITAR UNO"	indexOf + splice	splice(pos,1)
"TEXTO SEPARADO"	split	texto.split(';')
"UNIR LINEAS"	join	lineas.join("\n")

12 errores que debes revisar antes de entregar

1. Usar = en vez de ===.
2. Recorrer catalogo cuando el enunciado dice "todos los productos del pedido"; debes recorrer pedido.lineas.
3. Comparar producto.stock > 0 cuando la cantidad solicitada puede ser mayor: debe ser stock >= cantidad.
4. Confundir find con findIndex.
5. Usar map para filtrar.
6. Usar forEach esperando un array nuevo.
7. Ordenar numeros con sort() sin comparador.
8. Ordenar la entrada directamente cuando la funcion debe ser inmutable.
9. Olvidar Number() al convertir cantidades que vienen de split().
10. Confundir split (string -> array) con join (array -> string).
11. Olvidar el valor inicial de reduce cuando conviene o cuando el array puede estar vacio.
12. Usar delete en arrays: deja un hueco y no reajusta length; para quitar posiciones usa splice.

CHECKLIST DE 60 SEGUNDOS: 1) ¿He recorrido el array correcto? 2) ¿El metodo coincide con el verbo del enunciado? 3) ¿Tengo que devolver array, elemento, indice, booleano o un unico valor? 4) ¿Estoy mutando el original sin querer? 5) ¿Necesito ===? 6) ¿He convertido con Number? 7) ¿He puesto return donde toca? 8) ¿En sort de numbers he usado a-b/b-a? 9) ¿En texto he usado localeCompare? 10) ¿He protegido find frente a undefined?

FICHA FINAL: LECTURA DE CUALQUIER ENUNCIADO

Ejemplo: "Devuelve los nombres de los productos agotados ordenados alfabeticamente, sin modificar el catalogo".

```
const resultado = catalogo .filter(p => p.stock === 0) .map(p => p.nombre) .sort((a, b) => a.localeCompare(b))
```

Traduccion: agotados -> filter; nombres -> map; alfabeticamente -> sort + localeCompare. Como se pide no modificar el catalogo, el sort se aplica al array nuevo producido por map.

Ejemplo: "Comprueba que todos los productos pedidos existen y tienen stock suficiente".

```
return pedido.lineas.every(linea => { const producto = buscarProducto(catalogo, linea.nombre) return producto !== undefined && producto.stock >= linea.cantidad })
```

Traduccion: todos -> every; producto -> buscarProducto/find; existe -> !== undefined; stock suficiente -> stock >= cantidad.

Ejemplo: "Calcula el valor total del almacen".

```
return catalogo.reduce((total, p) => total + p.precio * p.stock, 0)
```

Traduccion: total -> reduce; valor de cada elemento -> precio * stock; valor inicial -> 0.

Miku: si dudas en el examen, vuelve siempre a esta pregunta: "Que pregunta del enunciado tengo que contestar?" La respuesta suele decirte el metodo.

O. EJEMPLOS DE LOS EJERCICIOS DE ARRAYS QUE YA HICIMOS

Playlist: lista de objetos {titulo, artista, duracion}. Si el objetivo es solo imprimir, usa forEach; si quieres una lista nueva, usa filter/map.

```
canciones.forEach(c => console.log(`${c.titulo} - ${c.artista}`))
```

Canciones largas: primero seleccionas las de mas de 180 segundos y despues las conviertes a texto.

```
const largas = canciones .filter(c => c.duracion > 180) .map(c => `${c.titulo} - ${c.duracion}s`) // ejemplo de resultado: ['Song A - 210s', 'Song C - 195s']
```

Biblioteca digital: un array de objetos {id,titulo,autor,paginas}. Para obtener el total de paginas, reduce; para buscar/eliminar, find + findIndex + splice.

```
const paginas = libros.reduce((total, libro) => total + libro.paginas, 0) const pos = libros.findIndex(libro => libro.id === 4) if (pos !== -1) libros.splice(pos, 1)
```

Validar: some/every sirven para preguntas globales sobre el array.

```
const hayLargos = libros.some(l => l.paginas > 500) const todosValidos = libros.every(l => typeof l.paginas === 'number' && l.paginas > 0)
```

Empleados: filter por departamento, reduce para media salarial y sort para ordenar por salario.

```
const delMarketing = empleados.filter(e => e.departamento === 'Marketing') const media = empleados.reduce((t,e) => t + e.salario, 0) / empleados.length const ordenados = [...empleados].sort((a,b) => b.salario - a.salario)
```

Lo que debes ver: en un mismo ejercicio se combinan metodos; cada uno hace una sola tarea y el siguiente recibe su resultado.

P. EJERCICIO 2.2 - PARTES 1 A 3: PATRONES QUE YA CONTROLAS

Apartado 2.2	Que pide	Patron	Salida/idea
1.1 crearCatalogo	Matriz -> objetos	map	[{nombre,categoria,precio,stock}, ...]
1.2 ampliarCatalogo	Catalogo nuevo + altas	concat	array nuevo
1.3 nombresOrdenados	Nombres alfabeticos	map + sort + localeCompare	[Ana,Zelda]
1.4 ordenarPorPrecio	Copia + orden precio	[...array].sort((a,b)=>...)	orden asc/desc
1.5 tresMasBaratos	3 menores	slice(0,3) + map	3 nombres
2.1 buscarProducto	Un producto, sin importar mayusculas	find + toLowerCase	objeto/undefined
2.2 existeProducto	Booleano usando map/includes	map + includes	true/false
2.3 posicionProducto	Indice de producto	findIndex	indice/-1
2.4 agotados	Nombres stock 0	filter + map	array de nombres
2.5 productosEntre	Precio dentro de rango	filter	array nuevo
3.1 valorAlmacen	Suma precio*stock	reduce	numero
3.2 productoMasCaro	Objeto con precio maximo	reduce	objeto
3.3 unidadesPorCategoria	Agrupar/contar	reduce + acumulador objeto	{cat: unidades,...}
3.4 hayAgotados	Existe stock 0	some	true/false
3.5 preciosValidos	Todos number y >0	every + typeof	true/false

Ejemplo 1.1: si la matriz contiene [Pan,Alimentacion,2,5], map debe producir { nombre:Pan, categoria:Alimentacion, precio:2, stock:5 }.

```
matriz.map(fila => ({ nombre: fila[0], categoria: fila[1], precio: fila[2], stock: fila[3] }))
```

Ejemplo 3.3: cuando reduce construye un objeto, piensa en "si no existe la categoria, inicializala; despues suma las unidades".

```
catalogo.reduce((acc, p) => { if (acc[p.categoria] === undefined) acc[p.categoria] = 0 acc[p.categoria] += p.stock return acc }, {})
```

Conexion con 4.2: both use every, but the array changes. En 3.5 recorres catalogo porque la pregunta habla de todos los precios del catalogo; en 4.2 recorres pedido.lineas porque la pregunta habla de todos los productos del pedido.

Q. FICHA DE CONSTRUCCION: DE DATOS A FUNCION

Paso 1 - Identifica la entrada: array, objeto, string, numero o varios parametros.

```
const funcion = (entrada) => { // ... }
```

Paso 2 - Decide la salida: array nuevo, objeto, elemento, indice, booleano, numero o string.

Salida	Metodos/patrones frecuentes	Ejemplo
Array nuevo	map/filter/slice/concat/flat	productos.map(p=>p.nombre)
Objeto/elemento	find/reduce	catalogo.find(p=>p.id===3)
Indice	findIndex/indexOf	carrito.indexOf(Zelda)
Booleano	some/every/includes	catalogo.some(p=>p.stock===0)
Numero	reduce	catalogo.reduce((t,p)=>t+p.precio,0)
String	join/template literal	lineas.join(\n)

Paso 3 - Pregunta sobre mutabilidad: ¿la funcion puede cambiar la entrada? En 2.2, salvo la Parte 5, las funciones no deben modificar los arrays de entrada.

Paso 4 - Haz la condicion exacta: usa === para igualdad, !== para desigualdad, >= cuando "suficiente" incluye el limite, && cuando deben cumplirse dos cosas.

```
producto !== undefined && producto.stock >= linea.cantidad
```

Paso 5 - Prueba mentalmente un caso: producto inexistente; stock 0; cantidad igual al stock; cantidad mayor al stock; array vacio; repetidos; dos elementos con el mismo nombre.

Resultado mental de algunos patrones:

```
[1,2,3].map(n => n*2) -> [2,4,6] [1,2,3].filter(n => n > 1) -> [2,3] [1,2,3].find(n => n > 1) -> 2 [1,2,3].findIndex(n => n > 1) -> 1 [1,2,3].some(n => n > 2) -> true [1,2,3].every(n => n > 0) -> true [1,2,3].reduce((t,n) => t+n,0) -> 6
```